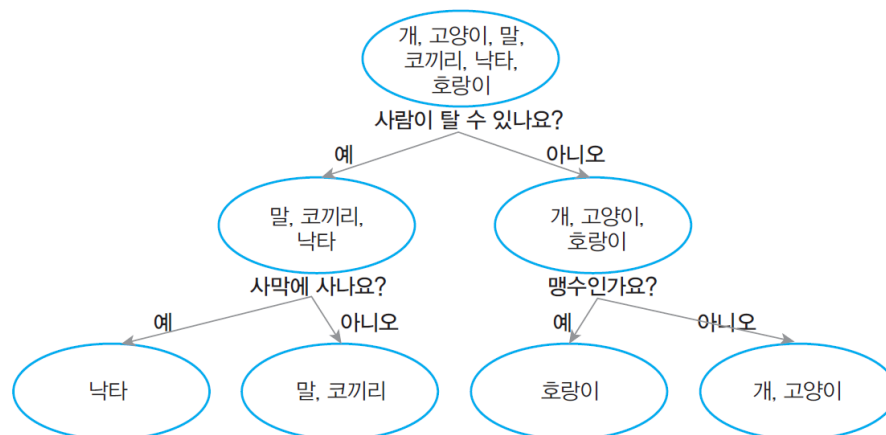


[Chap 8] 의사결정나무



스무고개 놀이

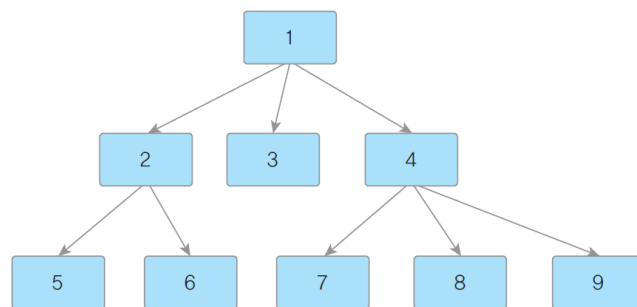
- 단순한 이진분류(binary classification)로 구성
 - 예 또는 아니오로 분기
- 의사결정나무 모델은 이와 동일한 개념



[그림 8-1] 스무고개 놀이

의사결정나무(Decision Tree)

- 의사결정 규칙을 나무 구조로 나타내어 전체 자료를 몇 개의 소집단으로 분류(classification)하거나 예측(prediction)을 수행하는 분석방법
- 주요 용어
 - 노드(node) 또는 마디
 - 그림에서 1에서 9까지의 숫자가 써 있는 것들
 - 루트 노드(root node)
 - 다른 곳에서 오지 않고 출발하기만 하는 노드 (그림의 1번)
 - 단말 노드(terminal node) 또는 리프 노드(leaf node)
 - 더 이상 분기되지 않는 노드 (그림의 5, 6, 7, 8, 9)
 - 부모 노드(parent node)
 - 자식 노드(child node)
- 가지 분기가 거듭될수록 그에 해당하는 데이터의 수가 줄어드는 구조



[그림 8-2] 의사결정나무 예시

3

의사결정나무 실습

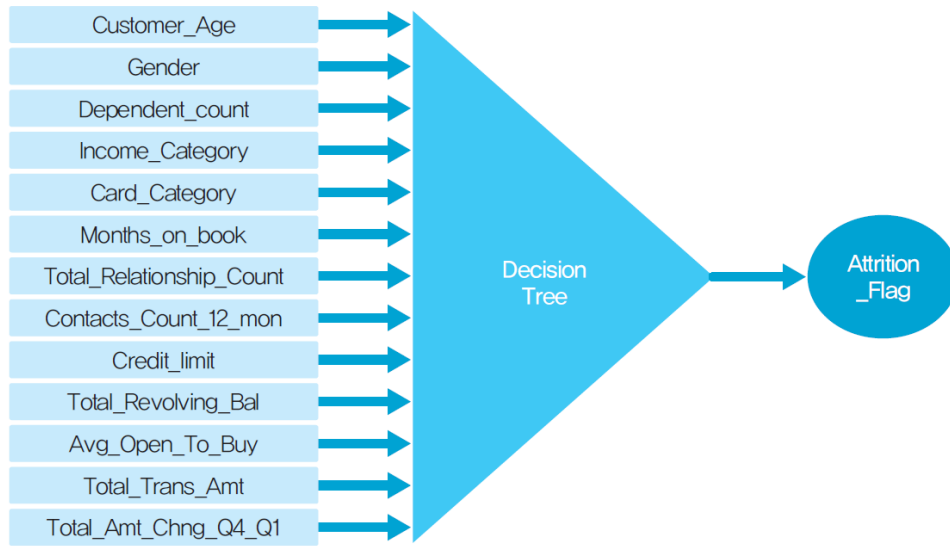
- 캐글(Kaggle)의 은행 신용카드 사용 고객 이탈여부 예측 데이터
 - bank_customer.csv

<표 8-1> 은행 신용카드 고객 데이터 열 설명

열 이름	설명
CLIENTNUM	고객번호
Attrition_Flag	이탈고객여부. 0=유지, 1=이탈
Customer_Age	고객 나이
Gender	고객 성별. 0=남성, 1=여성
Dependent_count	부양가족 수
Income_Category	연소득 수준. 0=자료없음, 1=\$40K 미만, 2=\$40K - 60K, 3=\$60K - \$80K, 4=\$80K-\$120K, 5=\$120K 초과
Card_Category	카드 등급. 0=Blue, 1=Silver, 2=Gold, 3=Platinum
Months_on_book	은행과의 총 거래기간
Total_Relationship_Count	은행 상품 보유 갯수
Contacts_Count_12_mon	최근 12개월간 은행과의 접촉 수
Credit_limit	신용 한도
Total_Revolving_Bal	리볼빙 잔액
Avg_Open_To_Buy	신용 잔여액
Total_Trans_Amt	평균 거래금액
Total_Amt_Chng_Q4_Q1	거래금액 변화량 (Q4 대비 Q1)

4

특성(feature)과 정답(target)



[그림 8-3] 은행 신용카드 고객 이탈 예측을 위한 모형

5

데이터 읽어오기

- bank_customer.csv를 읽어서 학습 데이터 구조 구성

```
1 import pandas as pd
2 df = pd.read_csv('bank_customer.csv')
3 feature = df[['Customer_Age', 'Gender', 'Dependent_count', 'Income_Category',
4   'Card_Category', 'Months_on_book', 'Total_Relationship_Count',
5   'Contacts_Count_12_mon', 'Credit_Limit', 'Total_Revolving_Bal',
6   'Avg_Open_To_Buy', 'Total_Trans_Amt', 'Total_Amt_Chng_Q4_Q1']]
7 target = df['Attrition_Flag']
```

- (3~6행) 데이터프레임에서 특성값으로 사용될 열들을 골라서 feature 변수에 넣음
- (7행) 정답 'Attrition_Flag'열을 target 변수에 넣음

6

학습 데이터와 테스트 데이터

- 데이터를 학습 데이터와 테스트 데이터로 나눔

```
1 from sklearn.model_selection import train_test_split
2 x_train, x_test, y_train, y_test = train_test_split(feature, target, test_size=0.2,
3 random_state=2222)
```

- (1행) sklearn.model_selection 패키지에서 train_test_split 함수를 들여옴
- (2~3행) train_test_split에 특성값 데이터셋 feature, 예측 대상 정답값 데이터셋 target을 넘겨주고, test_size=0.2로 하여 전체 데이터셋의 20%를 테스트 데이터로 사용하도록 함.
 - 전체 데이터 포인트 10127개 중 학습 데이터 8101개, 테스트 데이터 2026로 구성됨
 - random_state는 데이터를 랜덤하게 나눌 때 사용할 난수 값의 씨드값을 설정. 생략하면 실행할 때마다 학습 데이터와 테스트 데이터가 다르게 추출.

7

의사결정나무 모델 생성 및 예측

- 의사결정나무 모델을 생성하고, 학습용 데이터셋을 통해 학습시키고, 테스트용 데이터셋으로 예측을 수행

```
1 from sklearn.tree import DecisionTreeClassifier
2 dtc = DecisionTreeClassifier(random_state=2222)
3 dtc = dtc.fit(x_train,y_train)
4 y_pred = dtc.predict(x_test)
```

- (2행) DecisionTreeClassifier 모델을 생성하여 dtc라는 변수에 넣음
 - 의사결정나무 모델 초기화
 - 결과 비교를 위해 난수 씨드는 random_state=2222로 고정
- (3행) dtc에 저장되어 있는 초기 의사결정나무 모델에 학습용 데이터의 특성값 x_train와 정답값 y_train을 넘겨주고 기계학습을 수행
- (4행) 학습된 의사결정나무가 저장된 dtc에 테스트 데이터의 특성값인 x_test를 넣어서 예측

8

모델 예측 성능 확인

- 의사결정나무 모델의 예측 결과로 나온 y_{pred} 와 실제 값 y_{test} 값을 비교해 모델의 성능을 확인

```
1 from sklearn.metrics import confusion_matrix
2 from sklearn.metrics import classification_report
3 print(confusion_matrix(y_test, y_pred))
4 print(classification_report(y_test, y_pred))
```

```
[[1621  62]
 [ 79 264]]
```

		precision	recall	f1-score	support
	0	0.95	0.96	0.96	1683
	1	0.81	0.77	0.79	343
accuracy				0.93	2026
macro avg		0.88	0.87	0.87	2026
weighted avg		0.93	0.93	0.93	2026

- 정확도 0.93, F1-score 0.87로 준수한 성능을 보임
- 이탈 고객(정답값 1)에 대한 정밀도 0.81, 재현율 0.77, F1-score도 0.79로, 고객의 이탈을 상당한 수준으로 예측 가능

9

의사결정나무 시각화

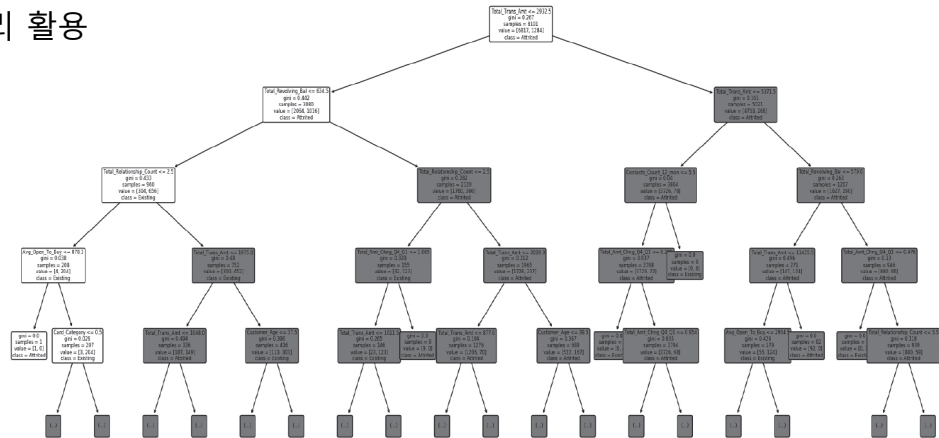
- 생성된 의사결정나무 모델을 직접 시각화해서 확인

```
1 from sklearn.tree import plot_tree
2 import matplotlib.pyplot as plt
3
4 plt.figure(figsize=(24,10))
5 plot_tree(dtc, feature_names=['Customer_Age', 'Gender',
6   'Dependent_count', 'Income_Category',
7   'Card_Category', 'Months_on_book', 'Total_Relationship_Count',
8   'Contacts_Count_12_mon', 'Credit_Limit', 'Total_Revolving_Bal',
9   'Avg_Open_To_Buy', 'Total_Trans_Amt', 'Total_Amt_Chng_Q4_Q1'],
10  class_names=['Attrited','Existing'], max_depth=4, rounded=True, fontsize=6)
11 plt.show()
```

- (4행) 전체 시각화 크기를 (24,10)으로 정함
- (5~10행) plot_tree()를 사용해 의사결정나무 모델을 시각화
 - 의사결정나무 모델 dtc를 인자로 넘겨주고, 표시될 특성값 이름들은 feature_names에 리스트로 정해줌
 - class_names에서는 예측 결과값 0과 1에 대응하는 제목을 'Existing'과 'Attrited'로 함
 - max_depth는 화면에 표시될 의사결정나무의 깊이를 정함
 - 노드 모양은 끝이 약간 둥글게 rounded=True로, 글씨 크기는 6포인트로 함

시각화 결과

- 각 노드에는 분기 기준이 쓰여져 있음
 - 루트 노드에서는 거래금액을 기준으로 2931.5보다 작으면 왼쪽, 아니면 오른쪽으로 분기하도록 함
 - 왼쪽으로 가보면 다시 리볼빙 잔액이 634.5보다 작은지에 따라 분기
 - 조건에 따라 분기해서 가지를 따라가면서 최종적으로 고객 이탈 여부를 판별
- 의사결정나무는 설명 가능(explainable)하다는 장점
 - 분기 기준을 사람이 보면 이해할 수 있음
 - 실무에서 널리 활용



[그림 8-4] 의사결정나무 시각화